# Course Overview: What Is Deep Learning?

Last modified: 2026-10-25

```
# The Python chunks on this page import these; reticulate provisions them.
# Attaching reticulate gives the inline R its `py` object.
library(reticulate)
py_require(c("numpy", "pandas", "matplotlib"))
```

The order of topics in these notes follows Brian Hutchinson's CSCI 481/581, used with his permission. The handwritten-digit exercises, the history and the road ahead are new.

## 1 Welcome

### 1.1 Plan for this lecture

The syllabus covers how the course runs. This lecture is about what the course is for:

1. why hand-designed features[1] run out, shown on a dataset of handwritten digits;
2. what representation learning and deep learning are;
3. where deep learning sits within machine learning[2] and artificial intelligence[3];
4. a short history of neural networks;
5. the topics and tools of these notes.

## 2 Features by hand

### 2.1 A first dataset: handwritten digits

The Optical Recognition of Handwritten Digits data [1] are scans of digits written by hand on forms. Each scan was reduced to an 8 by 8 grid of pixels, and each pixel counts how many of the 16 dots in its 4 by 4 block of a 32 by 32 bitmap of the scan were inked, so it holds a whole number from 0 to 16. The UCI Machine Learning Repository[4] publishes the data under a Creative Commons Attribution 4.0 license.

`data/optdigits-test.csv`[5] is a copy of its test file, `optdigits.tes`, byte for byte, under a new name. Each row is one image: 64 pixel values, read across the rows of the grid from top left to bottom right, then the digit the writer meant, 0 to 9. The file has no header row.

> **Exercise 2.1** (Loading the digits)**.** How many images of each digit does `data/optdigits-test.csv`[a] hold?
>
> [a] https://morrison-lab.github.io/dds/chapters/data/optdigits-test.csv

[1] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature
[2] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-machine-learning
[3] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-artificial-intelligence
[4] https://archive.ics.uci.edu/dataset/80
[5] https://morrison-lab.github.io/dds/chapters/data/optdigits-test.csv

Solution

*Solution* 2.1.

```
import numpy as np
import pandas as pd

digits = pd.read_csv("data/optdigits-test.csv", header=None)
print(digits.shape)
```

```
(1797, 65)
```

```
# Columns 0 to 63 are the pixels; column 64 is the digit.
pixels = digits.iloc[:, :64].to_numpy()
labels = digits[64].to_numpy()
digit_counts = np.bincount(labels)
print(digit_counts)
```

```
[178 182 177 183 181 182 181 179 174 180]
```

The file holds 1797 images, between 174 and 183 of each digit.

```
import matplotlib.pyplot as plt

# Row r of the file, read as an 8 by 8 grid, is image r.
images = pixels.reshape(-1, 8, 8)

fig, axes = plt.subplots(2, 10, figsize=(8, 2))
for k in range(10):
    for row in range(2):
        ax = axes[row, k]
        ax.imshow(images[labels == k][row], cmap="gray_r", vmin=0, vmax=16)
        ax.set_xticks([])
        ax.set_yticks([])
    axes[0, k].set_title(str(k))
plt.show()
```

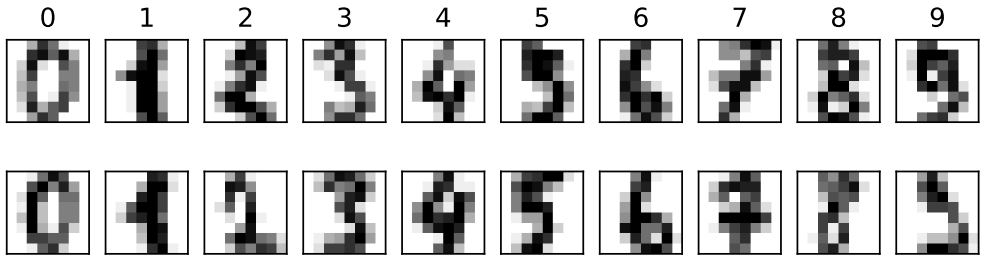



Figure 1: The first two images of each digit in the data.

Remark (One image, two arrangements)

*Remark* 2.1 (One image, two arrangements). The file stores each image as a row of 64 numbers, a feature vector[a] in $\mathbb{R}^{64}$, the form that models such as logistic regression[b] take as input. Figure 1 draws the same 64 numbers as an 8 by 8 grid: with rows and columns counted from 0, the pixel in row $i$ and column $j$ is entry $8i + j$ of the vector. `pixels.reshape(-1, 8, 8)` turns the rows of the file into grids, and nothing about the numbers changes. Models that look at pixels' neighbors, such as convolutional networks, keep the grid.

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature-vector
[b] https://morrison-lab.github.io/lds/chapters/logistic-regression.html

## 2.2 Writing a feature by hand

**Exercise 2.2** (A feature for 0 or 1)**.** Using the 0s and 1s in Figure 1, write a feature[a]: one number, computed from an image's 64 pixels, that tends to be small for a 0 and large for a 1.

**Hint**

Where does a 0 have no ink?

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

Solution

*Solution* 2.2. A 0 is a loop around an empty middle, and a 1 is a stroke down the middle. One feature[a] is the average ink in the middle of the grid: rows 2 to 5 and columns 3 and 4, counting from 0, eight pixels in all.

```
center_ink = images[:, 2:6, 3:5].mean(axis=(1, 2))
# The average center ink of each digit, 0 to 9.
print(pd.Series(center_ink).groupby(labels).mean().round(1))
```

```
0     1.7
1    13.3
2     8.9
3     7.7
4    10.0
5     7.1
6     8.0
7     9.8
8    10.8
9     6.6
dtype: float64
```

The 0s average 1.7 and the 1s 13.3, on the 0 to 16 scale of a pixel.

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

**Exercise 2.3** (A rule for 0 or 1)**.** Call an image a 1 when its center ink, from Solution 2.2, is above a cutoff, and a 0 otherwise. Find the cutoff that gets the fewest images of 0s and 1s wrong, with code or, on the web page, with the interactive plot after this exercise.

The course website's version of this page has an interactive plot here, for choosing a cutoff on the center ink of the 0s and 1s.

Solution

*Solution* 2.3. Try every cutoff from 0 to 16 in steps of 0.5, count each one's mistakes, and keep the cutoff with the fewest.

```
def best_cutoff(feature, is_high):
    """The cutoff on `feature` that best predicts `is_high`, and its mistakes."""
    cutoffs = np.arange(0, 16.5, 0.5)
    mistakes = np.array([np.sum((feature > c) != is_high) for c in cutoffs])
    return cutoffs[mistakes.argmin()], mistakes.min()

is_01 = (labels == 0) | (labels == 1)
cut_01, wrong_01 = best_cutoff(center_ink[is_01], labels[is_01] == 1)
print(cut_01, wrong_01, is_01.sum())
```

```
6.0 8 360
```

With a cutoff of 6, the rule gets 8 of the 360 images of 0s and 1s wrong, 2.2%. One feature[a], chosen by looking at a few images, is enough to tell these two digits apart.

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

**Exercise 2.4** (A feature for 3 or 8)**.** Using the 3s and 8s in Figure 1, write a feature[a] that tends to be small for a 3 and large for an 8.

**Hint**

Which side of a 3 is open?

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

Solution

*Solution* 2.4. An 8 closes its loops on the left, and a 3 leaves the left open. One feature[a] is the average ink on the left of the middle rows: rows 2 to 5 and columns 0 to 2. Search its cutoffs as Solution 2.3 did, to see how well it works.

```
left_ink = images[:, 2:6, 0:3].mean(axis=(1, 2))

is_38 = (labels == 3) | (labels == 8)
cut_38, wrong_38 = best_cutoff(left_ink[is_38], labels[is_38] == 8)
print(cut_38, wrong_38, is_38.sum())
```

```
2.0 40 357
```

With a cutoff of 2, the rule gets 40 of the 357 images of 3s and 8s wrong, 11.2%, against 2.2% for the 0s and 1s. The left ink of the 3s and 8s overlaps more than the center ink of the 0s and 1s, so this pair needs a better feature, or more than one.

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

## 2.3 All ten digits

**Exercise 2.5** (How many pairs?)**.** Telling all ten digits apart one pair at a time, as Exercise 2.2 and Exercise 2.4 did, takes a feature[a] for every pair of digits. How many pairs of digits are there?

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

Solution

*Solution* 2.5. A pair can start with any of the 10 digits and end with any of the other 9, and that counts each pair twice, once in each order:

$$\begin{aligned} \text{number of pairs} &= \frac{10 \times 9}{2} && \text{(first digit, second digit, divided by the 2 orders of a pair)} \\ &= \frac{90}{2} && \text{(multiply)} \\ &= 45 && \text{(divide)} \end{aligned}$$

That is 45 features[a] to design and test by hand, and Solution 2.4 shows that some pairs need more than one.

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

**Exercise 2.6** (Two features for ten digits)**.** Describe each image by the two features[a] so far, its center ink and its left ink. Classify each image as the digit whose average pair of features, over all the images of that digit, is nearest to the image's own pair. What share of the images does this classifier get right?

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

Solution

*Solution* 2.6.

```
def nearest_mean_accuracy(features, labels):
    """Share of images whose nearest digit average is their own digit."""
    means = np.array([features[labels == k].mean(axis=0) for k in range(10)])
    # Squared distance from every image (rows) to every digit's average (columns).
    distances = ((features[:, None, :] - means[None, :, :]) ** 2).sum(axis=2)
    return np.mean(distances.argmin(axis=1) == labels)

hand_features = np.column_stack([center_ink, left_ink])
acc_hand = nearest_mean_accuracy(hand_features, labels)
print(round(acc_hand, 3))
```

```
0.467
```

The two features[a] get 46.7% of the images right, so more than half of the images are still misread. This share is measured on the same images the digit averages were computed from, so it is a training accuracy, and new images would likely do a little worse.

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

**Exercise 2.7** (Better than guessing?)**.** How does the share of images that the classifier of Exercise 2.6 gets right compare with the share that always guessing the most common digit gets right?

Solution

*Solution* 2.7.

```
acc_guess = np.bincount(labels).max() / len(labels)
print(round(acc_guess, 3))
```

```
0.102
```

Guessing the most common digit gets 10.2% right, against 46.7% for the two features[a] (Solution 2.6), so the features carry real information. Doing better this way means designing more features, and features designed for digits do not carry over to faces, speech or text.

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

**Definition 2.1** (Feature engineering)**. Feature engineering** is designing by hand the features[a] that a model takes as input, using what a person knows about the problem [2, Ch. 1].

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

**Example 2.1** (Center ink and left ink)**.** Center ink (Solution 2.2) and left ink (Solution 2.4) are engineered features[a], which other sources also call hand-designed or hand-crafted features. Each one encodes a fact a person noticed about the shapes of digits: a 0 is empty in the middle, so its center ink averages 1.7 against 13.3 for a 1, and a 3 is open on the left, so its left ink averages 0.9 against 3.6 for an 8.

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

# 3 Learning the features

## 3.1 Representation learning

**Exercise 3.1** (Who chooses the features?)**.** In Exercise 2.6 a person chose the features[a], and the data chose only the digit averages. What else could choose the features?

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

Solution

*Solution* 3.1. The data could. A learning algorithm already chooses a model's weights to fit the data, and it can choose the features[a] the same way, as the hidden layer of a neural network does in lds's neural networks chapter[b].

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature
[b] https://morrison-lab.github.io/lds/chapters/neural-nets.html#sec-learning-the-features

**Definition 3.1** (Representation learning (feature learning))**. Representation learning**, or **feature learning**, is machine learning[a] in which the algorithm learns from data the features[b] a model computes from its raw input, as well as how the model maps those features to its output [2, Ch. 1].

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-machine-learning
[b] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

**Example 3.1** (Two learned features for the digits)**.** Principal component analysis[a] (PCA) learns features[b] from data with no labels: it finds the directions in the 64-dimensional space of pixel vectors along which the images vary most, and an image's features are its coordinates along those directions [2, Sec. 2.12]. Give the classifier of Exercise 2.6 the first two of these coordinates in place of center ink and left ink:

```
centered = pixels - pixels.mean(axis=0)
# The rows of vt are the directions of greatest variance, largest first.
_, _, vt = np.linalg.svd(centered, full_matrices=False)
learned_features = centered @ vt[:2].T
acc_learned = nearest_mean_accuracy(learned_features, labels)
print(round(acc_learned, 3), round(acc_hand, 3))
```

```
0.618 0.467
```

The two learned features get 61.8% of the images right, against 46.7% for the two engineered ones, both measured on the images they were learned from, and nobody told PCA what a digit looks like.

[a] https://morrison-lab.github.io/lds/chapters/unsupervised-learning.html#sec-pca
[b] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

## 3.2 Deep learning

**Exercise 3.2** (Features of features)**.** lds calls a neural network deep when its depth[a] gives it more than one hidden layer. Each hidden layer computes its features[b] from the layer before it. What could a second layer of features describe in a digit more easily than a first layer, computed straight from the pixels?

[a] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-depth
[b] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

Solution

*Solution* 3.2. A second layer can build features[a] out of features. A first layer might respond to short strokes in particular places; a second can combine strokes into loops, corners and lines; and the output layer can combine those into digits. Each layer describes the image in larger parts than the layer before it [2, Ch. 1].

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

**Definition 3.2** (Deep learning)**. Deep learning** is representation learning with models that compose many layers of learned features[a], each layer computed from the one before it [2, Ch. 1].

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

**Example 3.2** (AlexNet)**.** AlexNet, a network for sorting photographs into 1000 classes, has eight layers with weights: five convolutional layers, then three fully connected ones. Its 60 million parameters were learned from 1.2 million labeled images, and the 96 filters of its first layer learned on their own to respond to edges at different angles and to patches of color [3]. Nobody designed those features[a]; training found them.

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

Remark (Counting depth)

*Remark* 3.1 (Counting depth). These notes count a network's depth[a] as lds does, by its layers of weights, so AlexNet (Example 3.2) has depth 8. Other sources count only the hidden layers, or every layer including the input, and Goodfellow, Bengio and Courville measure depth as the longest path through the model's computational graph, noting that there is no single correct way to count it [2, Ch. 1]. "How deep is deep" has no agreed cutoff either.
Deep learning models are almost all neural networks[b]. Their name comes from early models of neurons, but the networks in these notes are not models of the brain.

[a] https://morrison-lab.github.io/lds/chapters/neural-nets.html#def-depth
[b] https://morrison-lab.github.io/lds/chapters/neural-nets.html

Watch

3Blue1Brown builds a network with two hidden layers for handwritten digits, and asks what features[a] its layers might learn, in But what is a neural network?[b] [4].

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

[b] https://www.youtube.com/watch?v=aircAruvnKk

Read

The introduction to *Deep Learning*, Chapter 1[a], explains representation learning and deep learning at more length [2, Ch. 1].

[a] https://www.deeplearningbook.org/contents/intro.html

## 3.3 Where deep learning fits

**Exercise 3.3** (Which programs are deep learning?)**.** Which of these programs are deep learning (Definition 3.2)?

- A spam filter whose rules a person wrote.
- The classifier of Exercise 2.6, on center ink and left ink.
- The same classifier on the two PCA features[a] of Example 3.1.
- AlexNet (Example 3.2).

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

Solution

*Solution* 3.3. Only AlexNet. The four fields of Figure 2 each contain the next, so all four programs are artificial intelligence[a], the last three are machine learning[b], the last two are representation learning, and the last is also deep learning:

- The hand-written spam filter follows rules and learns nothing from data.
- The classifier on center ink and left ink learns the digit averages from data, but a person chose its features[c].
- The classifier on PCA features learns its features too, but only one layer of them.
- AlexNet learns eight layers, each computed from the one before.

[a] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-artificial-intelligence
[b] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-machine-learning
[c] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

```
from matplotlib.patches import Ellipse

fields = [
    ("Artificial intelligence", "spam filter with hand-written rules"),
    ("Machine learning", "classifier on hand-designed features"),
    ("Representation learning", "PCA features"),
    ("Deep learning", "AlexNet"),
]
colors = ["#dbe9f6", "#a6cbe6", "#6aa6d6", "#2f7bbf"]

fig, ax = plt.subplots(figsize=(7, 4.5))
for depth, ((name, example), color) in enumerate(zip(fields, colors)):
    # Each ellipse sits inside the last, sharing its bottom edge.
    width, height = 10 - 2.2 * depth, 6.4 - 1.5 * depth
    ax.add_patch(Ellipse((5, height / 2), width, height,
                         facecolor=color, edgecolor="black"))
    text_color = "white" if depth == 3 else "black"
    ax.text(5, height - 0.6, name, ha="center", va="center",
            fontsize=11, fontweight="bold", color=text_color)
    ax.text(5, height - 1.0, example, ha="center", va="center",
            fontsize=9, style="italic", color=text_color)
ax.set_xlim(-0.1, 10.1)
ax.set_ylim(-0.1, 6.5)
ax.set_aspect("equal")
ax.set_axis_off()
plt.show()
```

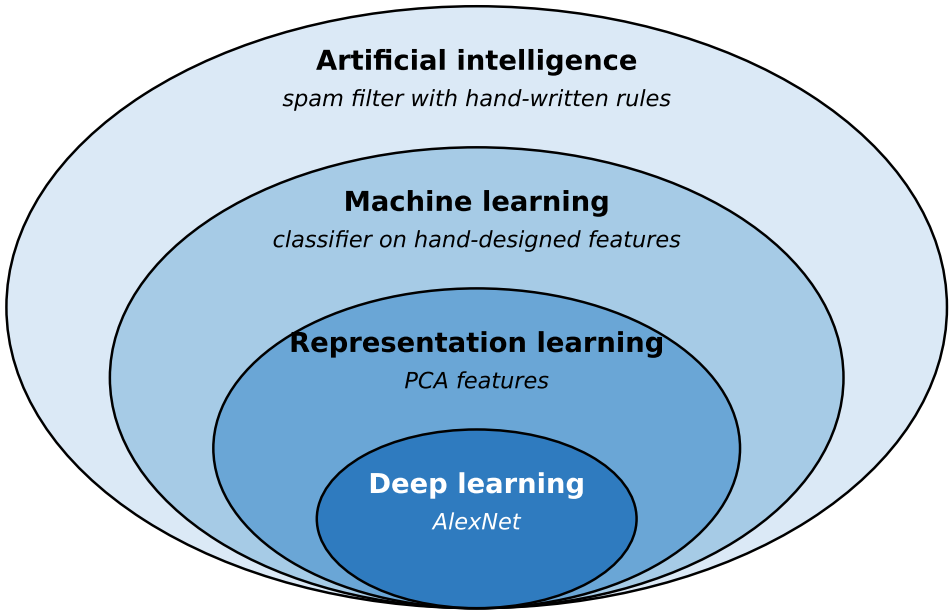



Figure 2: Deep learning is a kind of representation learning, which is a kind of machine learning[6], which is one approach to artificial intelligence[7]. After the Venn diagram in *Deep Learning* [2, Ch. 1], with this chapter's examples.

# 4 A short history

## 4.1 Three waves

Neural networks have risen in popularity three times, and fallen twice, under three names [2, Sec. 1.2.1]:

Table 1: The three waves of neural network research.

| Years | Name | Milestones |
|---|---|---|
| 1940s to 1960s | Cybernetics | A logical model of a neuron [5]; the perceptron, which learned its weights from examples [6] |
| 1980s to 1990s | Connectionism | Backpropagation for networks with hidden layers [7]; convolutional networks reading handwritten zip codes [8] |
| 2006 on | Deep learning | Deep belief networks, trained one layer at a time [9]; AlexNet [3]; the Transformer [10] |

The first wave faded after Minsky and Papert [11] showed that a perceptron, which has no hidden layer, cannot compute some simple functions of its inputs, such as the exclusive or of two binary inputs. Networks with hidden layers can, and backpropagation, one result of the second wave, trained them [2, Sec. 1.2.1], [7].

**Exercise 4.1** (Why did it take until 2012?)**.** Backpropagation for networks with hidden layers was published in 1986 [7], but deep networks only took over image recognition with AlexNet in 2012 [3]. What could have changed in those years?

Solution

*Solution* 4.1. Three things changed:

- **Data.** AlexNet learned from 1.2 million labeled images [3]; the zip-code network of 1989 learned from 7,291 digits [8]. Datasets grew this way across the field [2, Sec. 1.2.2].
- **Computing power.** AlexNet was trained on two graphics processing units (GPUs), chips built for the arithmetic of drawing images, which do many multiplications at once [3]. Faster hardware let networks grow larger [2, Sec. 1.2.3].
- **Techniques.** AlexNet used rectified linear units and dropout, which made a large network faster to train and less prone to overfitting [3].

The algorithm itself, gradient descent with backpropagation, changed little.

# 5 The road ahead

## 5.1 Topics

These notes cover these topics, in this order; the calendar says when each is taught.

- Tensors and automatic differentiation in PyTorch
- Feedforward networks, and backpropagation
- Optimization for deep networks
- Regularization, normalization and residual connections
- Convolutional networks
- Sequence models and language modeling
- Attention and Transformers
- Generative models: autoencoders, generative adversarial networks and diffusion
- Running deep learning experiments
- Applications, large language models, and ethics

### 5.2 Tools and prerequisites

The code in these notes, and in the homework, is Python with PyTorch [12], plus NumPy where a lecture builds a method from scratch, and pandas and matplotlib for reading data and drawing plots. Every example here runs in seconds on a laptop's CPU; the homework trains larger models.

These notes build on lds[8], the machine learning[9] notes for this course's prerequisite, and link to its definitions rather than repeating them. Its chapters on gradient descent[10], logistic regression[11], neural networks[12] and backpropagation[13] are the ones these notes lean on most.

## References


[1] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits." UCI Machine Learning Repository, 1998. doi: 10.24432/C50P49[14].

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: https://www.deeplearningbook.org/

[3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012. Available: https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks

[4] G. Sanderson, "But what is a neural network?" 3Blue1Brown, YouTube video, 2017. Available: https://www.youtube.com/watch?v=aircAruvnKk

[5] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *The Bulletin of Mathematical Biophysics*, vol. 5, no. 4, pp. 115–133, 1943, doi: 10.1007/BF02478259[15].

[6] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958, doi: 10.1037/h0042519[16].

[7] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986, doi: 10.1038/323533a0[17].

[8] Y. LeCun *et al.*, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989, doi: 10.1162/neco.1989.1.4.541[18].

[9] G. E. Hinton, S. Osindero, and Y.-W. Teh, "A fast learning algorithm for deep belief nets," *Neural Computation*, vol. 18, no. 7, pp. 1527–1554, 2006, doi: 10.1162/neco.2006.18.7.1527[19].

[10] A. Vaswani *et al.*, "Attention is all you need," in *Advances in neural information processing systems*, 2017. Available: https://arxiv.org/abs/1706.03762

[11] M. Minsky and S. Papert, *Perceptrons: An introduction to computational geometry*. Cambridge, MA: MIT Press, 1969.

[12] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in neural information processing systems*, 2019. Available: https://arxiv.org/abs/1912.01703


[8] https://morrison-lab.github.io/lds/
[9] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-machine-learning
[10] https://morrison-lab.github.io/lds/chapters/gradient-descent.html
[11] https://morrison-lab.github.io/lds/chapters/logistic-regression.html
[12] https://morrison-lab.github.io/lds/chapters/neural-nets.html
[13] https://morrison-lab.github.io/lds/chapters/backpropagation.html
[14] https://doi.org/10.24432/C50P49
[15] https://doi.org/10.1007/BF02478259
[16] https://doi.org/10.1037/h0042519
[17] https://doi.org/10.1038/323533a0
[18] https://doi.org/10.1162/neco.1989.1.4.541
[19] https://doi.org/10.1162/neco.2006.18.7.1527